ESTRUTURA DE DADOS · 2º SEMESTRE

Registro de Funcionários com Lista Simplesmente Encadeada

Explicação detalhada, trecho por trecho, do arquivo `Registro_Funcionarios.c`: o que cada parte faz, como funciona por dentro e por que foi escrita assim.

Autor do código: Aguinaldo Alves

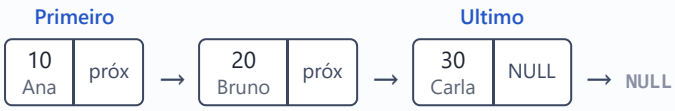
Data do código: 06/10/2026
Linguagem: C (Windows — usa `windows.h` e `conio.h`)
Tamanho: 901 linhas, 29 funções (incluindo `main`)
Versão da explicação: atualizada em 21/10/2026 — consulta com navegação P/V/S, funções `Ler_Arquivo` e `Gravar`, correção do `continue`

1 Visão geral do programa
2 Cabeçalho, bibliotecas e constante
3 As estruturas de dados
4 Funções de tela
5 Mensagens e perguntas no rodapé
6 Inicializar e contar
7 Pesquisa por código e por posição
8 Exibir e ler os dados
9 Criando um nó: `novo_item`
10 Inserções (final, início, posição)
11 Remoções (início, final, posição)
12 Tela de cadastro
13 Tela de remoção
14 Alteração e consulta com navegação
15 Arquivo: `Ler_Arquivo`, `Gravar`, liberar
16 Menu e função `main`
17 Resumo, complexidade e pontos de atenção

1. Visão geral do programa

O programa é um **sistema de cadastro de funcionários em modo texto** (console do Windows). Os funcionários ficam guardados na memória em uma **lista simplesmente encadeada**: cada funcionário mora em um "nó" (bloco de memória alocado com `malloc`), e cada nó sabe apenas o endereço do **próximo** nó. Por isso o nome "simplesmente" encadeada — só existe a ligação para frente.

A lista em si é uma pequena estrutura com dois ponteiros: `Primeiro` (onde a lista começa) e `Ultimo` (onde ela termina). O último nó aponta para `NULL`, que marca o fim.



Exemplo: lista com 3 funcionários. Cada caixa é um TipoItem: à esquerda o conteúdo (código e nome), à direita o campo proximo.

O que o usuário pode fazer

Opção	Ação	Função chamada
1 / 2 / 3	Cadastrar no final / no início / em uma posição	`cadastrar(&L, tipo)`
4 / 5 / 6	Remover do final / do início / de uma posição	`remover(&L, tipo)`
7	Alterar os dados de um funcionário	`alterar(&L)`
8	Consultar os funcionários, navegando com P (próximo), V (voltar) e S (sair)	`consultar(&L)`
9	Gravar a lista no arquivo e sair	`Gravar` + `liberar_lista`

Ciclo de vida dos dados

1. **Ao abrir:** a lista começa vazia e é preenchida com o que estiver no arquivo `funcionarios.dat`. Se o arquivo não existir, o programa avisa com uma mensagem de erro e começa com a lista vazia.
2. **Durante o uso:** todas as operações acontecem **só na memória** (nos nós da lista).
3. **Ao sair (opção 9):** a lista inteira é gravada de volta no arquivo e a memória é liberada com `free`.

Organização do código

O arquivo é organizado em camadas, das mais simples para as mais complexas — cada função só usa funções declaradas **acima** dela (em C, uma função precisa ser conhecida antes de ser chamada):

Camada	Funções
Tela	`gotoxy`, `telabordas`, `telafuncionario`, `limpar_mensagem`, `mensagem`, `perguntar`
Lista (operações básicas)	`inicializar_lista`, `contar_funcionarios`, `pesquisa`, `buscar_posicao`, `novo_item`, `inserir_*`, `remover_*`
Entrada/saída de dados	`exibir_funcionario`, `ler_texto`, `ler_funcionario`
Telas do sistema	`cadastrar`, `remover`, `alterar`, `consultar`, `menu`
Arquivo	`Ler_Arquivo`, `Gravar`, `liberar_lista`
Programa	`main`

O que mudou nesta versão

Esta explicação foi atualizada para acompanhar as mudanças feitas no código. Se você já leu a versão anterior, estes são os pontos novos:

Mudança	Antes	Agora	Onde ler
Consulta (opção 8)	O usuário digitava um código e via um funcionário por vez.	A tela já abre no **primeiro** funcionário e o usuário navega com **P** (próximo), **V** (voltar) e **S** (sair).	Seção 14
Leitura do arquivo	`carregar_arquivo`, ponteiro `FILE *arq`; arquivo ausente era ignorado em silêncio.	`Ler_Arquivo`, ponteiro `FILE *ptr`; arquivo ausente mostra uma **mensagem de erro** e o sistema inicia com a lista vazia.	Seção 15
Gravação do arquivo	`int salvar_arquivo` devolvia 1/0 e o `main` escrevia a mensagem.	`void Gravar` grava e ela mesma escreve a mensagem de sucesso ou de erro.	Seções 15 e 16
Bug do `continue`	Posição inválida na 1ª volta testava `resp` sem valor (lixo de memória).	`resp = 1;` antes do `continue` garante que a tela se repete.	Seções 12, 13 e 17
Nome do arquivo-fonte	`Registro_Funcionários.c` (com acento)	`Registro_Funcionarios.c` (sem acento, evita problemas no terminal e no compilador)	—

2. Cabeçalho, bibliotecas e constante

Registro_Funcionarios.c — linhas 1–27

```c
/*
    Autor: Aguinaldo Alves
    Data: 06/10/2026
    Objetivo: Criar um sistema para registro de funcionários utilizando o conceito de lista simplesmente encadeada.
        1 - Cadastrar funcionário no final da lista;
        2 - Cadastrar funcionário no início da lista;
        3 - Cadastrar funcionário em uma posição da lista;
        4 - Remover funcionário do final da lista;
        5 - Remover funcionário do início da lista;
        6 - Remover funcionário em uma posição da lista;
        7 - Alterar o cadastro de um funcionário;
        8 - Consultar os funcionários cadastrados;
        9 - Sair do programa (gravando a lista em disco).

    Persistência: ao iniciar, a lista é carregada do arquivo "funcionarios.dat";
    ao sair (opção 9), a lista é gravada novamente nesse arquivo.
*/

// Bibliotecas do C
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <conio.h>

// Definição de constantes
#define ARQUIVO "funcionarios.dat"
```

Comentário de cabeçalho (linhas 1–17)

Tudo entre `/*` e `*/` é um comentário de várias linhas: o compilador ignora. Ele serve como documentação: autor, data, objetivo e a lista das 9 operações. As linhas 15–16 explicam a **persistência** — como os dados sobrevivem depois que o programa fecha.

Bibliotecas (linhas 20–24)

A linguagem C, sozinha, é bem "pequena": ela tem `if`, `while`, `for`, variáveis, ponteiros e structs, mas **não tem nenhum comando embutido** para escrever na tela, ler o teclado, pedir memória ou abrir arquivos. Tudo isso vem de **bibliotecas** — coleções de funções prontas que acompanham o compilador.

Pense numa caixa de ferramentas: o `#include` é o momento em que você abre a gaveta certa. Tecnicamente, ele é uma instrução para o **pré-processador**, que "cola" o conteúdo do arquivo de cabeçalho (`.h`) no início do programa antes da compilação. Esse arquivo contém as **declarações** das funções: o nome, os parâmetros e o tipo de retorno de cada uma. Assim o compilador consegue conferir se você está chamando a função do jeito certo.

> **O que acontece se esquecer um `#include`?**
> O compilador encontra, por exemplo, `malloc(...)` e não sabe o que é. O GCC mostra um aviso como "*implicit declaration of function 'malloc*'" e passa a "adivinhar" que ela devolve `int`. Em alguns casos o programa até compila, mas pode funcionar errado (um ponteiro de 8 bytes cortado para um `int` de 4 bytes, por exemplo). Para tipos como `FILE` ou `COORD`, nem compila: "*unknown type name*". Por isso, **toda função de biblioteca usada precisa do seu `#include`**.

Abaixo, cada biblioteca do programa: o que ela oferece, onde é usada e **por que o programa não funcionaria sem ela**.

`#include <stdio.h>` — entrada e saída (*standard input/output*)

Por que foi necessária: é ela que permite ao programa **conversar com o usuário** e **usar arquivos** — as duas coisas mais básicas de um sistema de cadastro.

Recurso	Para que serve aqui
`printf`	Escrever toda a interface: molduras, rótulos, mensagens e os dados dos funcionários.
`scanf`	Ler números: opção do menu, código, posição, salário e as respostas "1 = Sim; 2 = Nao".
`fgets` e `stdin`	Ler textos com espaços (nome, endereço...) de forma segura, sem estourar o vetor.
`fflush`	Tentar limpar o "resto" do teclado antes de cada leitura (veja o ponto de atenção 1 na seção 17).
`FILE`, `fopen`, `fread`, `fwrite`, `fclose`	Abrir, ler, gravar e fechar o `funcionarios.dat` nas funções `Ler_Arquivo` e `Gravar`.

Sem ela: o programa não mostraria nada na tela, não leria nada do teclado e não conseguiria guardar os dados entre uma execução e outra.

`#include <stdlib.h>` — utilidades gerais (*standard library*)

Por que foi necessária: uma lista encadeada **cresce e diminui enquanto o programa roda**. Para isso é preciso pedir e devolver memória ao sistema operacional — e as funções que fazem isso estão aqui.

Recurso	Para que serve aqui
`malloc`	Em `novo_item`: reserva a memória de cada nó novo (um funcionário).
`free`	Nas remoções e em `liberar_lista`: devolve a memória do nó que saiu da lista.
`system`	Em `telabordas`: executa o comando `cls` do Windows, que limpa a tela.
`NULL`	O "endereço nenhum": marca o fim da lista e a lista vazia. (Ele também é definido em `stdio.h`.)

Sem ela: não haveria como criar nós — e sem nós não existe lista encadeada. Seria preciso usar um vetor de tamanho fixo, que é justamente o que o trabalho quer evitar.

`#include <string.h>` — manipulação de textos

Por que foi necessária: por um único motivo, mas importante. Quando o `fgets` lê "Ana" + Enter, ele guarda `"Ana\n"` — o Enter vem junto. Se esse `'\n'` ficasse no nome, ele quebraria a linha na tela toda vez que o nome fosse exibido.

Recurso	Para que serve aqui
`strcspn`	Em `ler_texto`: encontra a posição do `'\n'` para trocá-lo por `'\0'` (fim do texto).

Sem ela: seria preciso escrever um laço à mão para procurar o `'\n'`. Funciona, mas é mais código para escrever e testar — usar a função pronta é mais simples e mais confiável.

`#include <windows.h>` — recursos do Windows

Por que foi necessária: o C padrão só sabe escrever "em sequência", uma linha após a outra, como numa máquina de escrever. Para desenhar uma **tela com moldura** e colocar cada campo no seu lugar, é preciso **mover o cursor** para qualquer ponto do console — e isso só o próprio Windows sabe fazer.

Recurso	Para que serve aqui
`COORD`	Struct com os campos `X` (coluna) e `Y` (linha) de uma posição na tela.
`GetStdHandle(STD_OUTPUT_HANDLE)`	Pega o "identificador" da janela do console onde o programa escreve.
`SetConsoleCursorPosition`	Move o cursor para a coordenada desejada.

Sem ela: a função `gotoxy` não existiria, e todas as telas sairiam "empilhadas", sem moldura e sem os campos alinhados na coluna 28.

`#include <conio.h>` — entrada direta pelo console (*console input/output*)

Por que foi necessária: o `scanf` só entrega o que foi digitado **depois do Enter**. Em vários momentos o programa precisa reagir a **uma única tecla**, na hora, sem Enter e sem mostrar a letra na tela.

Recurso	Para que serve aqui
`getch`	Em `mensagem`: pausa até o usuário apertar qualquer tecla ("Pressione uma tecla..."). Em `consultar` (**novo**): lê a tecla **P**, **V** ou **S** e reage na hora — é o que torna a navegação rápida.

Sem ela: o usuário teria que apertar uma letra **e** Enter a cada passo na consulta, e as pausas de "Pressione uma tecla" precisariam de Enter.

> **Resumo: biblioteca padrão × biblioteca do Windows**
> `stdio.h`, `stdlib.h` e `string.h` fazem parte do **C padrão** e existem em qualquer compilador, em qualquer sistema. Já `windows.h` e `conio.h` **não** são padrão: existem nos compiladores para Windows (MinGW, MSVC). Por isso este programa só compila no Windows — em Linux/macOS seria preciso adaptar `gotoxy`, `getch` e `system("cls")`.

Biblioteca	Padrão C?	Necessidade em uma frase
`stdio.h`	Sim	Tela, teclado e arquivo.
`stdlib.h`	Sim	Memória dinâmica para os nós (`malloc` / `free`) e limpar a tela.
`string.h`	Sim	Tirar o Enter (`'\n'`) dos textos lidos.
`windows.h`	Não	Posicionar o cursor (`gotoxy`).
`conio.h`	Não	Ler uma tecla sem Enter (`getch`).

Constante `ARQUIVO` (linha 27)

`#define ARQUIVO "funcionarios.dat"` cria um nome que o pré-processador troca pelo texto `"funcionarios.dat"` em todo lugar onde aparece. A vantagem: se um dia o nome do arquivo mudar, basta alterar **uma linha**. Como o caminho é relativo, o arquivo é criado na **pasta de onde o programa é executado** (por isso ele aparece em `output/`, junto do `.exe`).

3. As estruturas de dados

Esta é a parte mais importante para entender o resto do programa. São três estruturas, cada uma com um papel.

Registro_Funcionarios.c — linhas 29–41

```c
// Definição Estruturas de dados
typedef struct
{
    int cd_funcionario;
    char nm_funcionario[50];
    char endereco[50];
    char dt_nasc[10];
    char telefone[15];
    char cpf[15];
    char cargo[40];
    char depto[30];
    float salario;
} reg_funcionario;
```

`reg_funcionario` — os dados de uma pessoa

`typedef struct { ... } reg_funcionario;` cria um **novo tipo** chamado `reg_funcionario`. Ele agrupa, em um único bloco, todos os campos de um funcionário:

Campo	Tipo	Observação
`cd_funcionario`	`int`	Código único — é a "chave" usada nas pesquisas.
`nm_funcionario`	`char[50]`	Até 49 caracteres + o `'\0'` que termina toda string em C.
`endereco`	`char[50]`	Idem.
`dt_nasc`	`char[10]`	Guardada como texto (`dd/mm/aa` = 8 caracteres).
`telefone` , `cpf`	`char[15]`	Texto, pois podem ter pontos, traços e zeros à esquerda.
`cargo` / `depto`	`char[40]` / `char[30]`	Texto.
`salario`	`float`	Número com casas decimais.

Somando, cada `reg_funcionario` ocupa cerca de 220 bytes (o valor exato é dado por `sizeof(reg_funcionario)` , que pode incluir bytes de alinhamento).

Registro_Funcionarios.c — linhas 43–58

```c
// Ponteiro para um item (nó) da lista
typedef struct TipoItem *TipoApontador;

// Cada item guarda os dados do funcionário e o endereço do próximo item
typedef struct TipoItem
{
    reg_funcionario conteudo;
    TipoApontador proximo;
} TipoItem;

// A lista guarda o endereço do primeiro e do último item
typedef struct
{
    TipoApontador Primeiro;
    TipoApontador Ultimo;
} TipoLista;
```

`TipoApontador` — um "atalho" para ponteiro (linha 44)

`typedef struct TipoItem *TipoApontador;` diz: "*TipoApontador* é o mesmo que *ponteiro para struct TipoItem*". Repare que ele é declarado **antes** da própria `struct TipoItem` existir. Isso é permitido em C porque um ponteiro sempre tem o mesmo tamanho, então o compilador não precisa conhecer o conteúdo da struct ainda (é uma *declaração antecipada*).

`TipoItem` — o nó da lista (linhas 47–51)

Cada nó tem duas partes:

- `conteudo`: um `reg_funcionario` inteiro (os dados).
- `proximo`: o endereço do nó seguinte. É isso que "encadeia" a lista. No último nó vale `NULL`.

Aqui foi preciso escrever `typedef struct TipoItem` (com nome depois de `struct`) justamente porque a struct se refere a si mesma através de `TipoApontador`. Uma estrutura que contém um ponteiro para o próprio tipo é chamada de **estrutura autorreferenciada**.

`TipoLista` — a "cabeça" da lista (linhas 54–58)

A lista não guarda funcionários diretamente; ela guarda apenas **dois endereços**:

- `Primeiro`: por onde começamos a percorrer a lista.
- `Ultimo`: permite inserir no final **sem percorrer** a lista inteira.



Na memória: a variável L (TipoLista, dentro de main) guarda só Primeiro e Ultimo; os nós ficam espalhados no heap, ligados pelo campo proximo.

> **Lista vazia**
> Quando não há nenhum funcionário, `Primeiro == NULL` e `Ultimo == NULL`. Quase todas as funções de inserção/remoção têm um `if` especial para esse caso — fique atento a ele nas próximas seções.

4. Funções de tela

Estas funções não mexem na lista; elas só "desenham" a interface no console.

Registro_Funcionarios.c — linhas 60–67

```c
// Função para posicionar o cursor na tela
void gotoxy(int x, int y)
{
    COORD coord;
    coord.X = x;
    coord.Y = y;
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
}
```

`gotoxy(x, y)`

Move o cursor do console para a coluna `x` e a linha `y` (contando a partir de 0, no canto superior esquerdo). O próximo `printf` escreve a partir dali. Passo a passo:

1. `COORD coord;` — `COORD` é uma struct do Windows com dois campos, `X` e `Y`.
2. Os campos recebem a posição desejada.
3. `GetStdHandle(STD_OUTPUT_HANDLE)` devolve o "identificador" da janela de saída (o console).
4. `SetConsoleCursorPosition(...)` posiciona o cursor naquele console, na coordenada informada.

Registro_Funcionarios.c — linhas 69–104

```c
// Função para desenhar as bordas da tela
void telabordas()
{
    system("cls");

    gotoxy(01, 01);
    printf("+--------------------------------------------------------------------------------+");

    gotoxy(01, 02);
    printf("| UNICIVE");
    gotoxy(22, 02);
    printf("CADASTRO DE FUNCIONARIOS");
    gotoxy(61, 02);
    printf("Estrutura de Dados |\n");

    gotoxy(01, 03);
    printf("|                                                                                |");

    gotoxy(01, 04);
    printf("+--------------------------------------------------------------------------------+");

    for (int i = 5; i <= 21; i++)
    {
        gotoxy(01, i);
        printf("|
|");
    }

    gotoxy(01, 22);
    printf("+--------------------------------------------------------------------------------+");

    gotoxy(01, 23);
    printf("|                                                                                |");

    gotoxy(01, 24);
    printf("+--------------------------------------------------------------------------------+");
}
```

`telabordas()`

Desenha a "moldura" que aparece em todas as telas:

- `system("cls")` executa o comando `cls` do Windows, que limpa o console.
- Linhas 1–4: o cabeçalho com "UNICIVE", "CADASTRO DE FUNCIONARIOS" e "Estrutura de Dados". A linha 3 fica em branco porque cada tela escreve ali o seu título (ex.: "MENU PRINCIPAL").
- O `for` das linhas 90–94 desenha as bordas laterais das linhas 5 a 21 — a área de trabalho.
- Linhas 22–24: a barra de rodapé. A linha 23 é usada para **mensagens e perguntas**.

> **Curiosidade: o zero à esquerda em `01`, `02` ...**
>
> Em C, um número inteiro que começa com `0` é lido em **base 8 (octal)**. Como `01` a `07` valem a mesma coisa em octal e decimal, funciona. Mas `08` e `09` **não compilam** (8 e 9 não existem em octal) — por isso nas linhas 115 e 228 aparece `9`, e não `09`.

Registro_Funcionarios.c — linhas 106–129

```c
// Função para desenhar a tela com os campos do funcionário
void telafuncionario()
{
    telabordas();

    gotoxy(3, 5);
    printf("Codigo do Funcionario: ");
    gotoxy(3, 7);
    printf("1 - Nome.............: ");
    gotoxy(3, 9);
    printf("2 - Endereco.........: ");
    gotoxy(3, 11);
    printf("3 - Dt Nasc.(dd/mm/aa): ");
    gotoxy(3, 13);
    printf("4 - Telefone.........: ");
    gotoxy(3, 15);
    printf("5 - CPF..............: ");
    gotoxy(3, 17);
    printf("6 - Cargo............: ");
    gotoxy(3, 19);
    printf("7 - Departamento......: ");
    gotoxy(3, 21);
    printf("8 - Salario..........: ");
}
```

`telafuncionario()`

Chama `telabordas()` e depois escreve os **rótulos** dos campos na coluna 3, pulando uma linha entre eles (linhas 5, 7, 9, ... 21). Os valores digitados ou exibidos sempre ficam na **coluna 28**, logo depois dos pontinhos. Essa coluna 28 vai aparecer em várias funções (`exibir_funcionario` , `ler_texto` , `cadastrar` ...).

Simulação de como a tela fica (gerada a partir das coordenadas usadas no código):

```
+------------------------------------------------------------------------------+
| UNICIVE           CADASTRO DE FUNCIONARIOS              Estrutura de Dados |
|                     CADASTRO NO FINAL DA LISTA                               |
+------------------------------------------------------------------------------+
| Codigo do Funcionario:   _                                                   |
|                                                                              |
| 1 - Nome.............:                                                       |
|                                                                              |
| 2 - Endereco.........:                                                       |
|                                                                              |
| 3 - Dt Nasc.(dd/mm/aa):                                                      |
|                                                                              |
| 4 - Telefone.........:                                                       |
|                                                                              |
| 5 - CPF..............:                                                       |
|                                                                              |
| 6 - Cargo............:                                                       |
|                                                                              |
| 7 - Departamento......:                                                      |
|                                                                              |
| 8 - Salario..........:                                                       |
+------------------------------------------------------------------------------+
|     Digite 0 para sair...                                                    |
+------------------------------------------------------------------------------+
```

Tela de funcionário (telafuncionario), com o título que `cadastrar` escreve na linha 3.

5. Mensagens e perguntas no rodapé

Registro_Funcionarios.c — linhas 131–159

```c
// Função limpar mensagem
void limpar_mensagem()
{
    gotoxy(03, 23);
    printf("%76s", "");
}

// Função para exibir uma mensagem na linha de rodapé e esperar uma tecla
void mensagem(char *texto)
{
    limpar_mensagem();
    gotoxy(07, 23);
    printf("%s", texto);
    getch();
}

// Função para ler um número inteiro na linha de rodapé
int perguntar(char *texto)
{
    int resp = 0;

    limpar_mensagem();
    gotoxy(07, 23);
    printf("%s", texto);
    fflush(stdin);
    scanf("%d", &resp);

    return resp;
}
```

`limpar_mensagem()`

Vai até a linha 23, coluna 3, e imprime **76 espaços**. O truque está em `printf("%76s", "")`: o `%76s` imprime uma string ocupando no mínimo 76 posições, completando com espaços à esquerda. Como a string é vazia, sobram só os espaços — que "apagam" o que estava escrito no rodapé.

`mensagem(char *texto)`

Limpa o rodapé, escreve o texto recebido na coluna 7 e chama `getch()`, que **pausa** o programa até o usuário apertar qualquer tecla. Serve para avisos: "Funcionario cadastrado!", "Lista vazia...", etc.

O parâmetro `char *texto` é um ponteiro para o primeiro caractere da frase. Quando chamamos `mensagem("Lista vazia...")`, o C passa o endereço do texto literal.

`perguntar(char *texto)`

Parecida com `mensagem`, mas em vez de esperar uma tecla ela **lê um número inteiro** e devolve esse número. É usada em todas as perguntas do tipo "1 = Sim; 2 = Nao".

- `int resp = 0;` — começa com 0 para que, se o usuário digitar algo que não é número (e o `scanf` falhar), a função devolva 0 e não "lixo" de memória.
- `fflush(stdin);` — tenta descartar o que sobrou no buffer do teclado (por exemplo, o Enter da leitura anterior). Veja a observação na seção 17.

- `scanf("%d", &resp);` — o `&` passa o **endereço** de `resp`, para que o `scanf` consiga gravar o valor lido dentro dela.

> **Por que criar essas funções?**
> Sem elas, as 4 linhas "limpar rodapé → posicionar → escrever → ler" seriam repetidas dezenas de vezes no código. Agrupar código repetido em funções deixa o programa menor e mais fácil de corrigir.

6. Inicializar e contar

Registro_Funcionarios.c — linhas 161–181

```c
// Função para inicializar a lista vazia
void inicializar_lista(TipoLista *L)
{
    L->Primeiro = NULL;
    L->Ultimo = NULL;
}

// Função que conta quantos funcionários existem na lista
int contar_funcionarios(TipoLista *L)
{
    int total = 0;
    TipoApontador p = L->Primeiro;

    while (p != NULL)
    {
        total++;
        p = p->proximo;
    }

    return total;
}
```

`inicializar_lista(TipoLista *L)`

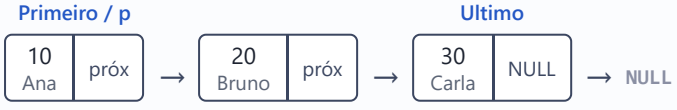
Coloca a lista no estado "vazia": `Primeiro` e `Ultimo` recebem `NULL`. É obrigatória antes de qualquer uso, pois uma variável local em C começa com **lixo de memória** — sem essa função, `L.Primeiro` poderia apontar para um endereço qualquer e o programa travaria.

Por que `TipoLista *L` (ponteiro) e a seta `->`? A função precisa alterar a variável `L` que está no `main`. Se recebesse `TipoLista L` (sem `*`), receberia uma **cópia** e as alterações se perderiam ao sair. Com o ponteiro ela altera o original. A seta é um atalho: `L->Primeiro` é o mesmo que `(*L).Primeiro`.

`contar_funcionarios(TipoLista *L)`

Percorre a lista do início ao fim contando os nós. Este é o **padrão de percurso** que aparece em quase todo o programa:

1. `p = L->Primeiro` — um ponteiro auxiliar começa no primeiro nó.
2. `while (p != NULL)` — enquanto não chegou no fim...
3. `total++` — conta o nó atual;
4. `p = p->proximo` — "pula" para o próximo nó.



p começa em Primeiro e, a cada volta do while, avança uma caixa (p = p->proximo) até chegar em NULL. Resultado: total = 3.

Como a lista não guarda o total em lugar nenhum, contar exige visitar todos os nós: o tempo cresce proporcionalmente ao número de funcionários (complexidade **O(n)**).

7. Pesquisa por código e por posição

Registro_Funcionarios.c — linhas 183–205

```c
// Função pesquisar: procura o código e devolve o endereço do item (ou NULL se não achar).
// Se "posicao" não for NULL, também devolve a posição (1, 2, 3...) do funcionário na lista.
TipoApontador pesquisa(TipoLista *L, int codigo, int *posicao)
{
    TipoApontador p = L->Primeiro;
    int pos = 1;

    while (p != NULL)
    {
        if (p->conteudo.cd_funcionario == codigo)
        {
            if (posicao != NULL)
            {
                *posicao = pos;
            }
            return p;
        }
        p = p->proximo;
        pos++;
    }

    return NULL;
}
```

`pesquisa(L, codigo, posicao)`

Procura um funcionário pelo código e devolve o **endereço do nó** onde ele está, ou `NULL` se não encontrar. É usada para:

- evitar códigos repetidos no cadastro;
- achar o funcionário na alteração.

Funcionamento: percorre a lista igual a `contar_funcionarios`, mas a cada nó compara `p->conteudo.cd_funcionario` com o código procurado. Ao mesmo tempo, `pos` vai contando 1, 2, 3... Quando acha, devolve `p` imediatamente com `return` (o laço é interrompido).

O parâmetro opcional `int *posicao`

Uma função em C só pode devolver **um** valor com `return`. Para devolver também a posição, a função recebe o endereço de uma variável de quem chamou e escreve nela com `*posicao = pos` (parâmetro de saída, ou "passagem por referência").

- Quem quer saber a posição passa o endereço de uma variável: `pesquisa(L, codigo, &posicao)`.
- Quem não precisa passa `NULL`: `pesquisa(L, codigo, NULL)`. O `if (posicao != NULL)` evita escrever em um endereço inválido.

Mudança nesta versão

Na versão anterior, `consultar` pesquisava pelo código e usava `&posicao` para mostrar "Posicao na lista: N". Com a nova consulta por navegação (seção 14), ninguém mais pede a posição: `cadastrar` e `alterar` passam `NULL`. O parâmetro continua funcionando e pode ser útil no futuro (por exemplo, numa opção "ir direto para o código X"), mas hoje ele não é usado. Retirá-lo seria uma simplificação possível — e um bom exercício.

Registro_Funcionarios.c — linhas 207–219

```
207 // Função que devolve o endereço do item que está em uma posição (1 = primeiro)
208 TipoApontador buscar_posicao(TipoLista *L, int posicao)
209 {
210     TipoApontador p = L->Primeiro;
211     int i;
212
213     for (i = 1; (i < posicao) && (p != NULL); i++)
214     {
215         p = p->proximo;
216     }
217
218     return p;
219 }
```

`buscar_posicao(L, posicao)`

Devolve o endereço do nó que está na posição pedida (1 = primeiro). Começa em `Primeiro` e avança `posicao - 1` vezes. Exemplo: para a posição 3, o `for` roda com `i = 1` e `i = 2` (duas voltas), parando no terceiro nó.

A condição `(p != NULL)` no `for` é uma proteção: se pedirem uma posição maior que a lista, o laço para em `NULL` em vez de tentar acessar `NULL->proximo` (o que derrubaria o programa).

Por que existe essa função?

Em um vetor, chegamos à posição 3 direto com `v[2]`. Em lista encadeada não existe acesso direto: é preciso "andar" de nó em nó. As inserções e remoções em posição usam `buscar_posicao(L, posicao - 1)` para parar no nó **anterior** ao ponto de mudança.

8. Exibir e ler os dados

Registro_Funcionarios.c — linhas 221–242

```c
// Função para exibir na tela os dados de um funcionário já cadastrado
void exibir_funcionario(reg_funcionario func)
{
    gotoxy(28, 05);
    printf("%d", func.cd_funcionario);
    gotoxy(28, 07);
    printf("%s", func.nm_funcionario);
    gotoxy(28, 9);
    printf("%s", func.endereco);
    gotoxy(28, 11);
    printf("%s", func.dt_nasc);
    gotoxy(28, 13);
    printf("%s", func.telefone);
    gotoxy(28, 15);
    printf("%s", func.cpf);
    gotoxy(28, 17);
    printf("%s", func.cargo);
    gotoxy(28, 19);
    printf("%s", func.depto);
    gotoxy(28, 21);
    printf("%.2f", func.salario);
}
```

`exibir_funcionario(reg_funcionario func)`

Escreve cada campo na coluna 28, na mesma linha do rótulo desenhado por `telafuncionario`. Detalhes:

- O parâmetro é passado **por valor** (uma cópia da struct), o que é seguro porque a função só lê os dados.
- `%d` para o código (inteiro), `%s` para os textos, `%.2f` para o salário com 2 casas decimais.

Registro_Funcionarios.c — linhas 244–254

```c
// Função para ler um texto em uma linha da tela.
// Limpa o que estava escrito no campo e retira o '\n' que o fgets guarda no final.
void ler_texto(int y, char *destino, int tamanho)
{
    gotoxy(28, y);
    printf("%49s", "");
    gotoxy(28, y);
    fflush(stdin);
    fgets(destino, tamanho, stdin);
    destino[strcspn(destino, "\n")] = '\0';
}
```

`ler_texto(y, destino, tamanho)` — leitura segura de texto

1. `printf("%49s", "")` apaga o campo na tela (49 espaços), útil na alteração, quando há um valor antigo exibido.
2. `fgets(destino, tamanho, stdin)` lê a linha inteira, **incluindo espaços** (o `scanf("%s")` pararia no primeiro espaço, então "Rua das Flores" viraria só "Rua"). O `tamanho` impede que o texto ultrapasse o vetor: lê no máximo `tamanho - 1` caracteres e reserva 1 para o `'\0'`.

3. O `fgets` guarda também o `'\n'` do Enter. A linha `destino[strcspn(destino, "\n")] = '\0';` resolve isso: `strcspn` devolve a posição do primeiro `'\n'` (ou o tamanho da string, se não houver), e nessa posição colocamos o terminador `'\0'`.

Exemplo: digitando `Ana` + Enter, o vetor fica `'A' 'n' 'a' '\n' '\0'`; `strcspn` devolve 3, e `destino[3] = '\0'` deixa só `"Ana"`.

Registro_Funcionarios.c — linhas 256–272

```c
// Função para ler os dados do funcionário (menos o código)
void ler_funcionario(reg_funcionario *func)
{
    ler_texto(7, func->nm_funcionario, 50);
    ler_texto(9, func->endereco, 50);
    ler_texto(11, func->dt_nasc, 10);
    ler_texto(13, func->telefone, 15);
    ler_texto(15, func->cpf, 15);
    ler_texto(17, func->cargo, 40);
    ler_texto(19, func->depto, 30);

    gotoxy(28, 21);
    printf("%49s", "");
    gotoxy(28, 21);
    fflush(stdin);
    scanf("%f", &func->salario);
}
```

`ler_funcionario(reg_funcionario *func)`

Lê todos os campos, **menos o código** (que é lido antes, por quem chama, para poder verificar duplicidade). Recebe um **ponteiro** porque precisa preencher a struct de quem chamou. Os textos usam `ler_texto` com a linha da tela e o tamanho de cada vetor; o salário, por ser `float`, é lido com `scanf("%f", &func->salario)`.

Note que `func->nm_funcionario` já é um endereço (nome de vetor vira ponteiro para o primeiro elemento), por isso não leva `&`. Já `func->salario` é um `float` comum e precisa de `&`.

9. Criando um nó: `novo_item`

Registro_Funcionarios.c — linhas 274–286

```c
// Função que cria um novo item na memória com os dados do funcionário
TipoApontador novo_item(reg_funcionario func)
{
    TipoApontador novo = (TipoApontador) malloc(sizeof(TipoItem));

    if (novo != NULL)
    {
        novo->conteudo = func;
        novo->proximo = NULL;
    }

    return novo;
}
```

Esta é a função que **cria memória nova** para cada funcionário. Linha a linha:

1. `malloc(sizeof(TipoItem))` pede ao sistema um bloco do tamanho exato de um nó (dados + ponteiro). O `malloc` devolve o endereço do bloco, ou `NULL` se não houver memória.
2. `(TipoApontador)` é um *cast*: converte o `void *` devolvido pelo `malloc` para o tipo de ponteiro certo.
3. `if (novo != NULL)` — só preenche se a alocação deu certo.
4. `novo->conteudo = func;` — copia a struct inteira de uma vez (em C, atribuir uma struct copia todos os campos, inclusive os vetores de `char`).
5. `novo->proximo = NULL;` — o nó nasce "solto", sem apontar para ninguém. Quem o insere na lista é que vai ligá-lo.



Resultado de novo_item: um nó isolado no heap, com proximo = NULL. Ele ainda NÃO faz parte da lista.

Memória dinâmica: por que não um vetor?
Um vetor tem tamanho fixo definido na compilação. Com `malloc`, cada funcionário novo pede só a memória de que precisa, e a lista pode crescer enquanto houver memória. Em compensação, cada `malloc` precisa de um `free` correspondente — o que é feito nas remoções e em `liberar_lista`.

10. Inserções na lista

As três funções de inserção devolvem `1` (sucesso) ou `0` (sem memória), para que a tela possa avisar o usuário.

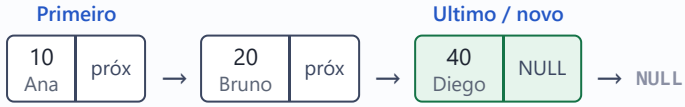
Registro_Funcionarios.c — linhas 288–309

```
288 // Inserir no final: o último item passa a apontar para o novo
289 int inserir_final(TipoLista *L, reg_funcionario func)
290 {
291     TipoApontador novo = novo_item(func);
292
293     if (novo == NULL)
294     {
295         return 0; // Sem memória
296     }
297
298     if (L->Primeiro == NULL) // Lista vazia: o novo é o primeiro e o último
299     {
300         L->Primeiro = novo;
301     }
302     else
303     {
304         L->Ultimo->proximo = novo;
305     }
306     L->Ultimo = novo;
307
308     return 1;
309 }
```

`inserir_final`

Graças ao ponteiro `Ultimo`, inserir no final é imediato — não precisa percorrer a lista:

- **Lista vazia** (`Primeiro == NULL`): o novo nó vira o primeiro.
- **Lista com itens**: o atual último passa a apontar para o novo (`L->Ultimo->proximo = novo`). Leia da esquerda para a direita: "o campo *proximo* do nó apontado por *Ultimo*".
- **Em ambos os casos** o novo vira o `Ultimo` (linha 306, fora do `if/else`).



Depois de inserir_final(Diego): Bruno (antigo último) aponta para Diego, e Ultimo passa a ser Diego.

Registro_Funcionarios.c — linhas 311–330

```c
// Inserir no início: o novo aponta para o antigo primeiro
int inserir_inicio(TipoLista *L, reg_funcionario func)
{
    TipoApontador novo = novo_item(func);

    if (novo == NULL)
    {
        return 0;
    }

    novo->proximo = L->Primeiro;
    L->Primeiro = novo;

    if (L->Ultimo == NULL) // Lista estava vazia
    {
        L->Ultimo = novo;
    }

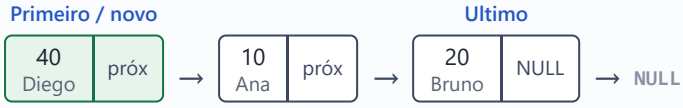
    return 1;
}
```

`inserir_inicio`

A ordem das duas linhas principais importa muito:

1. `novo->proximo = L->Primeiro;` — o novo nó aponta para quem era o primeiro;
2. `L->Primeiro = novo;` — a lista passa a começar no novo.

Se a ordem fosse invertida, perderíamos o endereço do antigo primeiro e toda a lista ficaria inacessível (vazamento de memória). Se a lista estava vazia, `L->Primeiro` era `NULL`, então o novo recebe `proximo = NULL` e também deve virar o `Ultimo` (linhas 324–327).



Depois de inserir_inicio(Diego): Diego aponta para Ana e passa a ser o Primeiro. Ultimo não muda.

Registro_Funcionarios.c — linhas 332–361

```c
// Inserir em uma posição (1 até total + 1)
int inserir_posicao(TipoLista *L, reg_funcionario func, int posicao)
{
    TipoApontador anterior;
    TipoApontador novo;

    if (posicao == 1)
    {
        return inserir_inicio(L, func);
    }

    if (posicao == contar_funcionarios(L) + 1)
    {
        return inserir_final(L, func);
    }

    novo = novo_item(func);
    if (novo == NULL)
    {
        return 0;
    }

    // Anda até o item que fica ANTES da posição desejada
    anterior = buscar_posicao(L, posicao - 1);

    novo->proximo = anterior->proximo;
    anterior->proximo = novo;

    return 1;
}
```

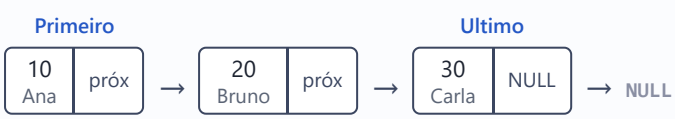
`inserir_posicao`

Aceita posições de `1` até `total + 1` (a validação é feita antes, em `cadastrar`). Primeiro trata as pontas reaproveitando funções prontas:

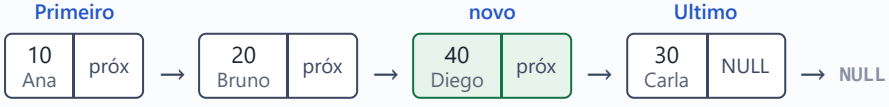
- `posicao == 1` → é uma inserção no início;
- `posicao == total + 1` → é uma inserção no final (e assim o `Ultimo` é atualizado corretamente).

Para o meio da lista, a ideia é parar no nó **anterior** à posição e "encaixar" o novo entre ele e o seguinte:

1. `anterior = buscar_posicao(L, posicao - 1);`
2. `novo->proximo = anterior->proximo;` — o novo aponta para quem vinha depois do anterior;
3. `anterior->proximo = novo;` — o anterior passa a apontar para o novo.



Antes: inserir Diego na posição 3. anterior = buscar_posicao(L, 2) = Bruno.



Depois: Bruno → Diego → Carla. Primeiro e Ultimo não mudam.

Mais uma vez, a ordem dos passos 2 e 3 é essencial: se fizéssemos o passo 3 antes, `anterior->proximo` já seria o próprio novo e perderíamos o caminho até Carla.

11. Remoções da lista

As funções de remoção supõem que a lista **não está vazia** e que a posição é válida — essas verificações são feitas antes, na função `remover` (seção 13). Todas terminam com `free`, que devolve a memória do nó ao sistema.

Registro_Funcionarios.c — linhas 363–376

```c
// Remover do início: o segundo item passa a ser o primeiro
void remover_inicio(TipoLista *L)
{
    TipoApontador p = L->Primeiro;

    L->Primeiro = p->proximo;

    if (L->Primeiro == NULL) // A lista ficou vazia
    {
        L->Ultimo = NULL;
    }

    free(p);
}
```

`remover_inicio`

1. Guarda o endereço do primeiro em `p` (precisamos dele para o `free` depois).
2. `L->Primeiro = p->proximo;` — a lista passa a começar no segundo nó.
3. Se o novo `Primeiro` é `NULL`, a lista tinha um único item e ficou vazia: `Ultimo` também vira `NULL`.
4. `free(p)` libera o nó removido.



remover_inicio: Primeiro passa para Bruno e o nó de Ana é liberado com free.

Registro_Funcionarios.c — linhas 378–400

```c
// Remover do final: precisa achar o penúltimo, pois a lista só anda para frente
void remover_final(TipoLista *L)
{
    TipoApontador anterior;

    if (L->Primeiro == L->Ultimo) // Só existe um item
    {
        free(L->Primeiro);
        L->Primeiro = NULL;
        L->Ultimo = NULL;
        return;
    }

    anterior = L->Primeiro;
    while (anterior->proximo != L->Ultimo)
    {
        anterior = anterior->proximo;
    }

    free(L->Ultimo);
    anterior->proximo = NULL;
    L->Ultimo = anterior;
}
```

`remover_final`

Aqui aparece a **principal limitação** da lista simplesmente encadeada: depois de remover o último, o **penúltimo** precisa virar o novo último (e ter `proximo = NULL`). Mas os nós só apontam para frente — o último não sabe quem está antes dele. Por isso é preciso percorrer desde o início:

- **Caso de um único item** (`Primeiro == Ultimo`): libera e deixa a lista vazia.
- **Caso geral**: o `while (anterior->proximo != L->Ultimo)` avança até o nó cujo próximo é o último — ou seja, o penúltimo. Depois: libera o último, corta a ligação (`anterior->proximo = NULL`) e atualiza `Ultimo`.



remover_final: o while para em Bruno (penúltimo). Carla é liberada, Bruno passa a apontar para NULL e vira o Ultimo.

Registro_Funcionarios.c — linhas 402–425

```c
// Remover de uma posição (1 até total)
void remover_posicao(TipoLista *L, int posicao)
{
    TipoApontador anterior;
    TipoApontador p;

    if (posicao == 1)
    {
        remover_inicio(L);
        return;
    }

    anterior = buscar_posicao(L, posicao - 1);
    p = anterior->proximo;

    anterior->proximo = p->proximo; // "Pula" o item removido

    if (p == L->Ultimo)
    {
        L->Ultimo = anterior;
    }

    free(p);
}
```

`remover_posicao`

Se a posição é 1, reaproveita `remover_inicio`. Caso contrário:

1. `anterior = buscar_posicao(L, posicao - 1)` — para no nó antes do que será removido;
2. `p = anterior->proximo` — `p` é o nó que sai;
3. `anterior->proximo = p->proximo` — o anterior "pula" o removido e passa a apontar para o seguinte;
4. Se o removido era o último, o `anterior` vira o novo `Ultimo`;
5. `free(p)`.



remover_posicao(L, 2): anterior = Ana. Ana passa a apontar direto para Carla e o nó de Bruno é liberado.

Por que guardar `p` antes de mudar as ligações?

Depois de `anterior->proximo = p->proximo`, nenhum nó da lista aponta mais para o removido. Se não tivéssemos o endereço salvo em `p`, seria impossível chamar `free` — a memória ficaria perdida (*memory leak*).

12. Tela de cadastro: `cadastrar(L, tipo)`

Uma única função atende às opções 1, 2 e 3 do menu. O parâmetro `tipo` diz onde inserir (1 = final, 2 = início, 3 = posição). Isso evita escrever três funções quase iguais.

Registro_Funcionarios.c — linhas 427–470

```
427 // Função de cadastro. tipo: 1 = final; 2 = início; 3 = posição
428 void cadastrar(TipoLista *L, int tipo)
429 {
430     // Declaração de variáveis
431     reg_funcionario func;
432     TipoApontador result;
433     int posicao = 0;
434     int total;
435     int resp;
436
437     do
438     {
439         telafuncionario();
440         gotoxy(24, 03);
441         if (tipo == 1)
442             printf("CADASTRO NO FINAL DA LISTA");
443         else if (tipo == 2)
444             printf("CADASTRO NO INICIO DA LISTA");
445         else
446             printf("CADASTRO EM UMA POSICAO DA LISTA");
447
448         // Para cadastro em posição, primeiro pergunta onde inserir
449         if (tipo == 3)
450         {
451             total = contar_funcionarios(L);
452             limpar_mensagem();
453             gotoxy(07, 23);
454             printf("Informe a posicao (1 a %d) ou 0 para sair: ", total + 1);
455             posicao = 0;
456             fflush(stdin);
457             scanf("%d", &posicao);
458
459             if (posicao == 0)
460             {
461                 return;
462             }
463
464             if ((posicao < 1) || (posicao > total + 1))
465             {
466                 mensagem("Posicao invalida... Pressione uma tecla.");
467                 resp = 1; // O continue vai direto para o while: força repetir o laço
468                 continue;
469             }
470         }
```

Preparação da tela (linhas 437–446)

Tudo fica dentro de um `do { ... } while (resp == 1)`: a tela se repete enquanto o usuário responder "1" à pergunta "Deseja cadastrar outro?". O título da linha 3 muda conforme o `tipo`.

Escolha da posição — só no tipo 3 (linhas 449–470)

- Mostra o intervalo válido: de 1 até `total + 1` (a posição `total + 1` significa "depois do último").
- `posicao = 0` antes do `scanf`: se o usuário digitar uma letra, o valor continua 0 e a função sai em vez de usar lixo.
- Digitar `0` sai da função com `return`, voltando ao menu.
- Posição fora do intervalo: mostra o erro, faz `resp = 1;` e executa `continue` (linhas 466–468).

> **Correção nesta versão: `resp = 1;` antes do `continue`**
>
> Dentro de um `do { ... } while (condição)`, o `continue` **não** volta direto para o início do laço: ele pula para a **verificação da condição** — aqui, `while (resp == 1)`. Só se a condição for verdadeira o laço recomeça.
>
> **O problema (versão anterior):** se o usuário errasse a posição logo na **primeira** volta, a pergunta "Deseja cadastrar outro?" ainda não tinha sido feita, então `resp` nunca tinha recebido valor. Variável local sem valor em C contém **lixo de memória** — um número qualquer. Às vezes a tela se repetia, às vezes o programa voltava para o menu sem explicação.
>
> **A solução:** colocar `resp = 1;` logo antes do `continue`. Agora a condição `resp == 1` é **garantidamente verdadeira**, e o laço sempre recomeça: a tela é redesenhada e a posição é pedida de novo. Pense assim: é como se o próprio programa respondesse "Sim, quero tentar de novo".

Registro_Funcionarios.c — linhas 472–501

```
472         limpar_mensagem();
473         gotoxy(07, 23);
474         printf("Digite 0 para sair...");
475
476         // Lê o código até digitar um que não exista (ou 0 para sair)
477         do
478         {
479             func.cd_funcionario = 0;
480             gotoxy(28, 05);
481             printf("%49s", "");
482             gotoxy(28, 05);
483             fflush(stdin);
484             scanf("%d", &func.cd_funcionario);
485
486             // Pesquisando se o código do funcionário já existe.
487             result = pesquisa(L, func.cd_funcionario, NULL);
488
489             if (result != NULL)
490             {
491                 limpar_mensagem();
492                 gotoxy(07, 23);
493                 printf("Codigo ja cadastrado... Digite outro ou 0 para sair.");
494             }
495
496         } while ((result != NULL) && (func.cd_funcionario != 0));
497
498         if (func.cd_funcionario == 0)
499         {
500             return;
501         }
```

Leitura do código sem repetição (linhas 477–496)

Um `do-while` interno insiste até o usuário digitar um código que **ainda não existe** (ou 0 para sair):

1. Apaga o campo do código (coluna 28, linha 5) e lê o número.
2. `pesquisa(L, codigo, NULL)`: se devolver um endereço (≠ `NULL`), o código já está em uso → mostra o aviso.
3. A condição `(result != NULL) && (codigo != 0)` repete enquanto o código for repetido **e** diferente de zero.

Ao sair do laço, há duas possibilidades: o código é novo (segue o cadastro) ou é 0 (linhas 498–501: `return`, volta ao menu).

Registro_Funcionarios.c — linhas 503–528

```
503         limpar_mensagem();
504         ler_funcionario(&func);
505
506         resp = perguntar("Deseja gravar o funcionario (1 = Sim; 2 = Nao).: ");
507
508         if (resp == 1)
509         {
510             int ok;
511
512             if (tipo == 1)
513                 ok = inserir_final(L, func);
514             else if (tipo == 2)
515                 ok = inserir_inicio(L, func);
516             else
517                 ok = inserir_posicao(L, func, posicao);
518
519             if (ok)
520                 mensagem("Funcionario cadastrado! Pressione uma tecla.");
521             else
522                 mensagem("Erro: memoria insuficiente... Pressione uma tecla.");
523         }
524
525         resp = perguntar("Deseja cadastrar outro (1 = Sim; 2 = Nao). : ");
526
527     } while (resp == 1);
528 }
```

Leitura dos dados e gravação na lista (linhas 503–523)

1. `ler_funcionario(&func)` preenche os demais campos. O `&` é necessário porque a função recebe um ponteiro.
2. Pergunta se deve gravar. Só com resposta 1 o funcionário entra na lista — qualquer outra resposta descarta os dados digitados (eles só existiam na variável local `func`).
3. Conforme o `tipo`, chama `inserir_final`, `inserir_inicio` ou `inserir_posicao`. A variável `ok` recebe 1 ou 0 e define a mensagem.

Repare na divisão de responsabilidades: `cadastrar` cuida da **conversa com o usuário** (validar, perguntar, avisar) e as funções `inserir_*` cuidam **apenas dos ponteiros**. Essa separação deixa cada parte mais simples de entender e testar.

13. Tela de remoção: `remover(L, tipo)`

Registro_Funcionarios.c — linhas 530–587

```c
// Função de remoção. tipo: 1 = final; 2 = início; 3 = posição
void remover(TipoLista *L, int tipo)
{
    // Declaração de variáveis
    TipoApontador p;
    int posicao;
    int total;
    int resp;

    do
    {
        telafuncionario();
        gotoxy(24, 03);
        if (tipo == 1)
            printf("REMOCAO DO FINAL DA LISTA");
        else if (tipo == 2)
            printf("REMOCAO DO INICIO DA LISTA");
        else
            printf("REMOCAO EM UMA POSICAO DA LISTA");

        total = contar_funcionarios(L);

        if (total == 0)
        {
            mensagem("Lista vazia... Pressione uma tecla.");
            return;
        }

        // Descobre qual posição será removida
        if (tipo == 1)
        {
            posicao = total;
        }
        else if (tipo == 2)
        {
            posicao = 1;
        }
        else
        {
            limpar_mensagem();
            gotoxy(07, 23);
            printf("Informe a posicao (1 a %d) ou 0 para sair: ", total);
            posicao = 0;
            fflush(stdin);
            scanf("%d", &posicao);

            if (posicao == 0)
            {
                return;
            }

            if ((posicao < 1) || (posicao > total))
            {
                mensagem("Posicao invalida... Pressione uma tecla.");
                resp = 1; // O continue vai direto para o while: força repetir o laço
                continue;
```

```
586             }
587         }
```

Mesma ideia de `cadastrar`: uma função para as opções 4, 5 e 6, com o `tipo` indicando de onde remover.

Proteção contra lista vazia (linhas 550–556)

Antes de qualquer coisa, conta os funcionários. Se for 0, avisa e sai. Essa verificação é o que garante que `remover_inicio`, `remover_final` e `remover_posicao` nunca recebam uma lista vazia (elas acessariam `NULL->proximo` e o programa travaria).

Transformar o tipo em uma posição (linhas 558–587)

Para poder **mostrar** o funcionário antes de remover, a função descobre a posição dele:

- Tipo 1 (final) → posição = `total`;
- Tipo 2 (início) → posição = `1`;
- Tipo 3 → pergunta ao usuário, aceitando de 1 a `total` (aqui não existe `total + 1`, pois não dá para remover um item que não existe).

Posição inválida: a mesma correção de `cadastrar` foi aplicada aqui (linhas 583–585) — `resp = 1;` antes do `continue` garante que a tela de remoção se repita, em vez de depender de lixo de memória.

Registro_Funcionarios.c — linhas 589–610

```
589         // Mostra o funcionário que será removido
590         p = buscar_posicao(L, posicao);
591         exibir_funcionario(p->conteudo);
592 
593         resp = perguntar("Confirma a remocao (1 = Sim; 2 = Nao).      : ");
594 
595         if (resp == 1)
596         {
597             if (tipo == 1)
598                 remover_final(L);
599             else if (tipo == 2)
600                 remover_inicio(L);
601             else
602                 remover_posicao(L, posicao);
603 
604             mensagem("Funcionario removido! Pressione uma tecla.");
605         }
606 
607         resp = perguntar("Deseja remover outro (1 = Sim; 2 = Nao).    : ");
608 
609     } while (resp == 1);
610 }
```

Exibir, confirmar e remover (linhas 589–608)

1. `p = buscar_posicao(L, posicao)` pega o nó e `exibir_funcionario(p->conteudo)` mostra os dados — o usuário vê **quem** vai ser removido.
2. Pede confirmação. Só com resposta 1 chama a função de remoção correspondente ao `tipo`.
3. Pergunta se quer remover outro, e o `do-while` recomeça.

Boa prática de interface

Mostrar o registro e pedir confirmação antes de uma ação que apaga dados evita remoções por engano — principalmente nas opções "remover do início/final", em que o usuário não escolhe o funcionário diretamente.

14. Alteração e consulta

Registro_Funcionarios.c — linhas 612–667

```c
void alterar(TipoLista *L)
{
    // Declaração de variáveis
    reg_funcionario func;
    TipoApontador p;
    int resp;

    do
    {
        telafuncionario();
        gotoxy(31, 03);
        printf("ALTERACAO");

        gotoxy(07, 23);
        printf("Digite 0 para sair...");

        func.cd_funcionario = 0;
        gotoxy(28, 05);
        fflush(stdin);
        scanf("%d", &func.cd_funcionario);

        if (func.cd_funcionario == 0)
        {
            return;
        }

        // Pesquisando se o código do funcionário existe.
        p = pesquisa(L, func.cd_funcionario, NULL);

        if (p == NULL)
        {
            mensagem("Funcionario nao encontrado... Pressione uma tecla.");
        }
        else
        {
            // Mostra os dados atuais do funcionário encontrado
            exibir_funcionario(p->conteudo);

            mensagem("Pressione uma tecla para digitar os novos dados...");
            limpar_mensagem();

            ler_funcionario(&func);

            resp = perguntar("Deseja gravar a alteracao (1 = Sim; 2 = Nao).: ");

            if (resp == 1)
            {
                p->conteudo = func; // cd_funcionario continua o mesmo do pesquisado
                mensagem("Alteracao gravada! Pressione uma tecla.");
            }
        }

        resp = perguntar("Deseja alterar outro (1 = Sim; 2 = Nao).    : ");

    } while (resp == 1);
```

```
667 }
```

`alterar(TipoLista *L)`

1. Lê o código do funcionário (0 sai).
2. `p = pesquisa(L, codigo, NULL)`. Se `NULL`, avisa que não encontrou.
3. Se encontrou, mostra os dados atuais e espera uma tecla.
4. `ler_funcionario(&func)` lê os novos valores. Como `ler_texto` apaga cada campo antes de ler, o valor antigo some da tela conforme o usuário digita.
5. Se confirmar, `p->conteudo = func;` substitui **de uma vez** todos os dados dentro do nó.

Detalhe importante (linha 659): a struct `func` inteira é copiada para o nó, inclusive o campo `cd_funcionario`. Isso não estraga nada porque `func.cd_funcionario` foi preenchido no `scanf` da linha 631 com o código pesquisado — é o mesmo código que já estava no nó. O nó também não muda de lugar: as ligações (`proximo`) continuam intactas, porque só o campo `conteudo` foi alterado.

`consultar(TipoLista *L)` — navegação pela lista (versão nova)

Na versão anterior, para ver cada funcionário era preciso digitar o código dele — e, se você não lembrasse os códigos, não conseguia ver a lista. A nova consulta funciona como **folhear um fichário**: abre na primeira ficha, e você vai passando para a próxima (**P**) ou voltando para a anterior (**V**), até decidir sair (**S**).

Registro_Funcionarios.c — linhas 669–692

```
669 // Função de consulta: já abre no primeiro funcionário e permite navegar pela lista
670 // com P (próximo) e V (voltar), sem precisar digitar a posição de cada cadastro.
671 void consultar(TipoLista *L)
672 {
673     // Declaração de variáveis
674     TipoApontador p;
675     int posicao;
676     int total;
677     int tecla;
678
679     total = contar_funcionarios(L);
680
681     if (total == 0)
682     {
683         telafuncionario();
684         gotoxy(31, 03);
685         printf("CONSULTA");
686         mensagem("Lista vazia... Pressione uma tecla.");
687         return;
688     }
689
690     // Começa mostrando o primeiro funcionário da lista
691     p = L->Primeiro;
692     posicao = 1;
```

Parte 1 — preparação (linhas 674–692)

1. **Variáveis:** `p` é o "dedo" que aponta para o funcionário que está na tela; `posicao` diz em qual ficha estamos (1, 2, 3...); `total` guarda quantos funcionários existem; `tecla` guarda a tecla apertada.
2. **Lista vazia (linhas 679–688):** se `total == 0`, não há o que mostrar. A função desenha a tela, avisa "Lista vazia..." e sai com `return`. Essa verificação é **essencial**: sem ela, a linha seguinte faria `p = NULL` e o

`exibir_funcionario(p->conteudo)` tentaria ler dados do endereço `NULL` — o programa travaria.

3. **Ponto de partida (linhas 691–692):** `p = L->Primeiro` e `posicao = 1` — a consulta sempre começa na primeira ficha.

> **Por que contar o total só uma vez?**
> `contar_funcionarios` percorre a lista inteira. Como a consulta não cadastra nem remove ninguém, o total não muda enquanto ela está aberta. Então basta contar uma vez, no início, e reaproveitar o valor no rodapé "Reg. 3 de 15".

Registro_Funcionarios.c — linhas 694–707

```
694     do
695     {
696         // Redesenha a tela inteira para não sobrar texto do funcionário anterior
697         telafuncionario();
698         gotoxy(31, 03);
699         printf("CONSULTA");
700
701         exibir_funcionario(p->conteudo);
702
703         limpar_mensagem();
704         gotoxy(03, 23);
705         printf("Reg. %d de %d   P = Proximo   V = Voltar   S = Sair", posicao, total);
706
707         tecla = getch();
```

Parte 2 — mostrar e esperar uma tecla (linhas 694–707)

A cada volta do `do-while`:

1. `telafuncionario()` redesenha a tela **inteira** (moldura e rótulos). Isso é importante: se o funcionário anterior tinha um nome comprido e o atual tem um nome curto, sem redesenhar sobrariam letras do nome antigo no final da linha.
2. `exibir_funcionario(p->conteudo)` mostra os dados do nó para onde `p` aponta.
3. O rodapé mostra a posição e as teclas disponíveis: `Reg. %d de %d` recebe `posicao` e `total`.
4. `tecla = getch();` espera **uma** tecla, sem Enter (é para isso que a biblioteca `conio.h` foi incluída). O `getch` devolve o código da tecla como um número inteiro — por exemplo, `'P'` vale 80 e `'p'` vale 112 na tabela ASCII.



Simulação da tela de consulta mostrando o 1º de 15 funcionários (dados fictícios).

Registro_Funcionarios.c — linhas 709–740

```
709         switch (tecla)
710         {
711         case 'P':
712         case 'p':
713             if (p->proximo == NULL)
714             {
715                 mensagem("Este ja e o ultimo funcionario... Pressione uma tecla.");
716             }
717             else
718             {
719                 p = p->proximo;
720                 posicao++;
721             }
722             break;
723
724         case 'V':
725         case 'v':
726             if (posicao == 1)
727             {
728                 mensagem("Este ja e o primeiro funcionario... Pressione uma tecla.");
729             }
730             else
731             {
732                 // A lista só anda para frente: para voltar, percorre de novo desde o primeiro
733                 posicao--;
734                 p = buscar_posicao(L, posicao);
735             }
736             break;
737         }
738
739     } while ((tecla != 'S') && (tecla != 's'));
740 }
```

Parte 3 — decidir o que fazer com a tecla (linhas 709–739)

O `switch (tecla)` compara a tecla com cada `case`:

- `case 'P': case 'p':` — dois `case` seguidos, sem nada entre eles, levam ao **mesmo** código. Assim tanto P maiúsculo quanto minúsculo funcionam (o usuário não precisa se preocupar com o Caps Lock). Isso acontece porque, sem `break`, a execução "escorre" de um `case` para o seguinte.
- **Próximo (P):** se `p->proximo == NULL`, estamos no último nó e não há para onde ir — mostra um aviso. Senão, `p = p->proximo` (anda uma ficha) e `posicao++`. É exatamente o mesmo passo de percurso de `contar_funcionarios`, só que dado **um de cada vez**, a pedido do usuário.
- **Voltar (V):** se `posicao == 1`, já estamos no primeiro — mostra um aviso. Senão, diminui a `posicao` e chama `buscar_posicao(L, posicao)`.
- **Qualquer outra tecla** (inclusive S): nenhum `case` combina, o `switch` não faz nada e o programa segue para a condição do laço.

A condição `while ((tecla != 'S') && (tecla != 's'))` repete o laço enquanto a tecla **não** for S nem s. Ao apertar S, o laço termina, a função acaba e o programa volta ao menu. Uma tecla qualquer (como X) apenas redesenha o mesmo funcionário.

> **Por que o "voltar" é diferente do "próximo"?**
>
> Para avançar, basta seguir a seta: `p = p->proximo`. Mas cada nó só conhece o **próximo** — ninguém guarda o endereço do **anterior**. É a mesma limitação que aparece em `remover_final` (seção 11).
>
> A solução usada é "voltar ao começo e andar de novo": se estamos na posição 5 e queremos a 4, `buscar_posicao(L, 4)` parte do `Primeiro` e anda 3 passos. É simples e sempre correto, mas custa O(n): quanto mais longe na lista, mais passos para voltar uma ficha. Com 15 funcionários isso é instantâneo; com milhões, seria perceptível.



Exemplo: p está em Carla (posição 3). P → p = p->proximo (Diego, posição 4). V → buscar_posicao(L, 2) recomeça em Primeiro e anda 1 passo até Bruno.

Teste de mesa da consulta

Lista com 3 funcionários. Acompanhe o que acontece com as variáveis a cada tecla:

Tecla	O que acontece	`posicao`	`p` aponta para
(início)	Abre no primeiro	1	1º funcionário
V	Aviso "Este ja e o primeiro funcionario"	1	1º funcionário
p	Avança (minúscula também funciona)	2	2º funcionário
P	Avança	3	3º funcionário
P	Aviso "Este ja e o ultimo funcionario"	3	3º funcionário
V	Volta com `buscar_posicao(L, 2)`	2	2º funcionário
S	Sai do laço e volta ao menu	—	—

A consulta continua útil para testar o programa: depois de cadastrar no início, no final ou em uma posição, basta navegar com P para conferir se o funcionário realmente foi parar no lugar esperado — o rodapé "Reg. N de T" mostra a posição.

15. Arquivo: `Ler_Arquivo`, `Gravar` e liberar

A lista encadeada vive na **memória RAM**, que é apagada quando o programa fecha. Para os funcionários "sobreviverem" de uma execução para a outra, eles precisam ir para o **disco**, num arquivo. Nesta versão, as duas funções de arquivo foram reescritas no padrão usado em aula: os nomes `Ler_Arquivo` e `Gravar`, e o ponteiro de arquivo chamado `ptr`.

> **Antes de começar: o que é `FILE *ptr`?**
> `FILE` é uma struct da biblioteca `stdio.h` que guarda tudo o que o sistema precisa saber sobre um arquivo aberto (qual arquivo é, em que ponto da leitura estamos, etc.). Nós nunca mexemos nos campos dela diretamente: apenas guardamos o **endereço** que o `fopen` devolve em `ptr` e o entregamos para `fread`, `fwrite` e `fclose`. Pense em `ptr` como a "senha da fila": é com ela que as outras funções sabem de qual arquivo você está falando.

Modo do `fopen`	Significado	Se o arquivo não existe	Se o arquivo existe
`"rb"`	**r**ead (ler) + **b**inary (binário)	Devolve `NULL` (erro)	Abre para leitura, do início
`"wb"`	**w**rite (gravar) + **b**inary (binário)	Cria um arquivo novo	**Apaga** o conteúdo e começa do zero

Por que binário? No modo binário, os bytes da struct vão para o disco exatamente como estão na memória: cada funcionário ocupa sempre `sizeof(reg_funcionario)` bytes. Isso permite ler e gravar um funcionário inteiro com **uma única** chamada de `fread` / `fwrite`, sem precisar converter cada campo em texto. (A desvantagem é que o arquivo não pode ser lido num editor de texto.)

Registro_Funcionarios.c — linhas 742–771

```c
// Função que lê os funcionários do arquivo para a lista (ao abrir o programa)
// Modo "rb": leitura (r = read) de arquivo binário (b = binary)
void Ler_Arquivo(TipoLista *L)
{
    FILE *ptr;
    reg_funcionario func;

    ptr = fopen(ARQUIVO, "rb");

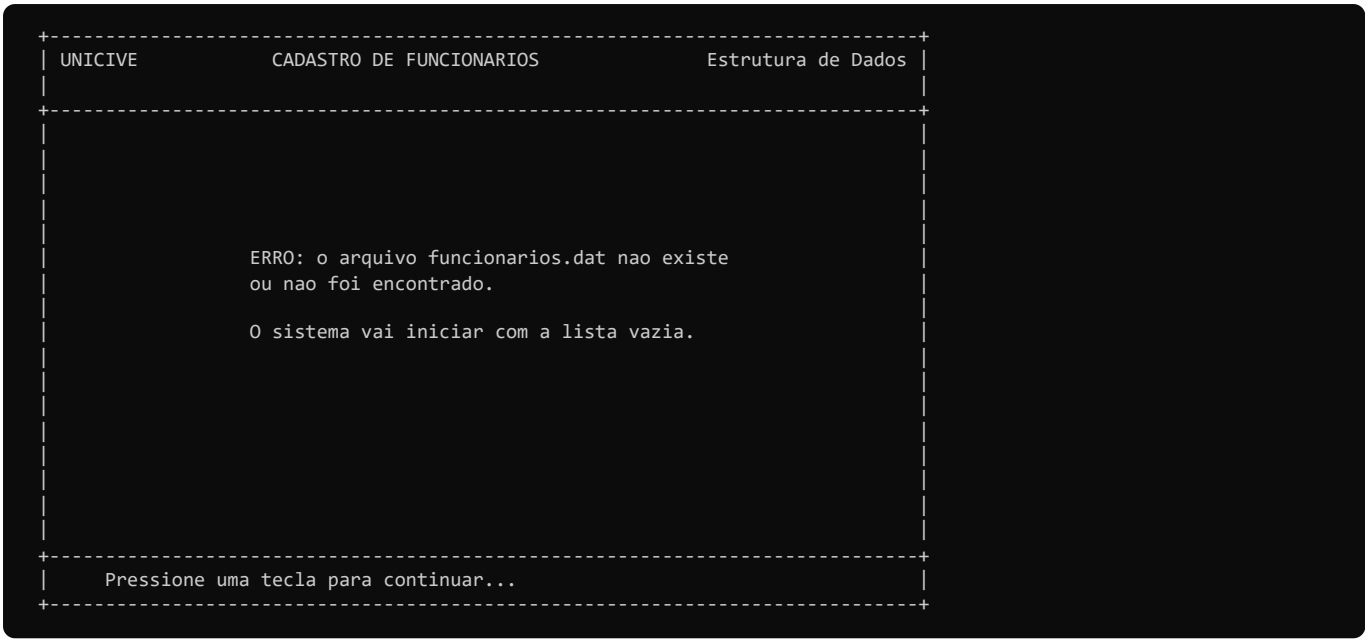
    if (ptr == NULL) // O arquivo não existe ou não foi encontrado
    {
        telabordas();
        gotoxy(20, 10);
        printf("ERRO: o arquivo %s nao existe", ARQUIVO);
        gotoxy(20, 11);
        printf("ou nao foi encontrado.");
        gotoxy(20, 13);
        printf("O sistema vai iniciar com a lista vazia.");
        mensagem("Pressione uma tecla para continuar...");
        return;
    }

    // Lê um registro por vez e coloca no final da lista, mantendo a ordem gravada
    while (fread(&func, sizeof(reg_funcionario), 1, ptr) == 1)
    {
        inserir_final(L, func);
    }

    fclose(ptr);
}
```

`Ler_Arquivo` — do disco para a memória (ao abrir o programa)

1. `ptr = fopen(ARQUIVO, "rb");` tenta abrir o `funcionarios.dat` para leitura binária.
2. **Arquivo não encontrado (linhas 751–762) — novidade desta versão.** Se `ptr == NULL`, o arquivo não existe (primeira execução) ou não está na pasta de onde o programa foi aberto. Antes, a função apenas retornava em silêncio. Agora ela desenha a moldura, mostra uma **mensagem de erro** explicando o que aconteceu e avisa que o sistema vai começar com a lista vazia. O `mensagem(...)` espera uma tecla, para o usuário ter tempo de ler. Depois o `return` encerra a função — **sem** chamar `fclose`, porque nenhum arquivo foi aberto.
3. **Leitura dos registros (linhas 765–768):** `fread(&func, sizeof(reg_funcionario), 1, ptr)` significa: "leia **1** bloco de `sizeof(reg_funcionario)` bytes do arquivo `ptr` e coloque no endereço de `func`". Ela devolve quantos blocos conseguiu ler: 1 enquanto houver dados, 0 quando o arquivo acaba — e é isso que encerra o `while`.
4. Cada registro lido é inserido com `inserir_final`, o que preserva a **mesma ordem** em que a lista foi gravada.
5. `fclose(ptr)` fecha o arquivo e libera os recursos do sistema.



Simulação da tela mostrada por `Ler_Arquivo` quando o `funcionarios.dat` não é encontrado.

> **Por que avisar em vez de ficar em silêncio?**
> Imagine que o `funcionarios.dat` está em outra pasta. Sem o aviso, o usuário veria "Funcionarios cadastrados: 0" e acharia que perdeu todos os dados. Pior: se cadastrasse alguém e saísse, `Gravar` criaria um arquivo novo só com esse funcionário. A mensagem deixa claro **o que aconteceu** e **o que o programa vai fazer**.

Repare que o arquivo guarda apenas os `reg_funcionario`, **nunca os ponteiros**. Endereços de memória mudam a cada execução, então gravá-los não faria sentido; a lista é reconstruída do zero, com nós novos.

Registro_Funcionarios.c — linhas 773–798

```c
// Função que grava todos os funcionários da lista no arquivo (ao sair do programa)
// Modo "wb": gravação (w = write) de arquivo binário (b = binary); apaga o conteúdo anterior
void Gravar(TipoLista *L)
{
    FILE *ptr;
    TipoApontador p;

    ptr = fopen(ARQUIVO, "wb");

    limpar_mensagem();
    gotoxy(03, 23);

    if (ptr == NULL)
    {
        printf("ERRO ao gravar %s! Programa finalizado.", ARQUIVO);
        return;
    }

    for (p = L->Primeiro; p != NULL; p = p->proximo)
    {
        fwrite(&p->conteudo, sizeof(reg_funcionario), 1, ptr);
    }

    fclose(ptr);
    printf("Dados gravados em %s. Programa finalizado.", ARQUIVO);
}
```

`Gravar` — da memória para o disco (ao sair)

1. `ptr = fopen(ARQUIVO, "wb");` — se o arquivo existir, ele é **apagado e recriado**. Assim, o arquivo sempre fica igual à lista atual (inclusive sem os funcionários removidos).
2. `limpar_mensagem()` e `gotoxy(03, 23)` preparam o rodapé para a mensagem final.
3. Se `ptr == NULL` (por exemplo, pasta sem permissão de escrita ou disco cheio), mostra "ERRO ao gravar" e sai.
4. O `for (p = L->Primeiro; p != NULL; p = p->proximo)` é o mesmo percurso de `contar_funcionarios`, escrito de forma compacta: as três partes do `for` são "começa no primeiro", "enquanto não chegar no fim" e "anda para o próximo".
5. `fwrite(&p->conteudo, sizeof(reg_funcionario), 1, ptr)` grava os bytes do conteúdo de cada nó. A expressão `&p->conteudo` significa "endereço do campo *conteudo* do nó apontado por *p*". Só o conteúdo é gravado — o campo `proximo` fica de fora.
6. `fclose(ptr)` — **muito importante** na gravação: o sistema costuma guardar os dados num *buffer* na memória e só escrevê-los de verdade no disco ao fechar. Sem o `fclose`, parte dos dados poderia não ser gravada.
7. Por fim, mostra "Dados gravados em funcionarios.dat. Programa finalizado.".

> **Mudança nesta versão: de `int salvar_arquivo` para `void Gravar`**
>
> Antes, a função devolvia 1 ou 0, e o `main` testava esse valor para escolher a mensagem. Agora `Gravar` é `void` (não devolve nada) e **ela mesma** escreve a mensagem de sucesso ou de erro. Resultado: o `case 9` do `main` ficou com uma única linha, e tudo o que diz respeito à gravação está num lugar só.

Registro_Funcionarios.c — linhas 800–807

```c
// Função que libera da memória todos os itens da lista
void liberar_lista(TipoLista *L)
{
    while (L->Primeiro != NULL)
    {
        remover_inicio(L);
    }
}
```

`liberar_lista`

Remove o primeiro nó repetidamente até a lista ficar vazia. Reaproveitar `remover_inicio` é uma escolha inteligente: é a remoção mais barata (O(1)) e já cuida de atualizar `Primeiro`, `Ultimo` e chamar `free`. Assim, toda memória pedida com `malloc` é devolvida antes de o programa terminar.

16. Menu e função `main`

Registro_Funcionarios.c — linhas 809–846

```c
int menu(TipoLista *L)
{
    int opcao = -1;

    telabordas();

    gotoxy(33, 03);
    printf("MENU PRINCIPAL");

    gotoxy(20, 6);
    printf("1 - Cadastrar Funcionario no Final da Lista");
    gotoxy(20, 7);
    printf("2 - Cadastrar Funcionario no Inicio da Lista");
    gotoxy(20, 8);
    printf("3 - Cadastrar Funcionario em uma Posicao da Lista");
    gotoxy(20, 10);
    printf("4 - Remover Funcionario no Final da Lista");
    gotoxy(20, 11);
    printf("5 - Remover Funcionario no Inicio da Lista");
    gotoxy(20, 12);
    printf("6 - Remover Funcionario em uma Posicao da Lista");
    gotoxy(20, 14);
    printf("7 - Alteracao do Cadastro de Funcionario");
    gotoxy(20, 15);
    printf("8 - Consultar Funcionarios");
    gotoxy(20, 17);
    printf("9 - Sair do Programa");

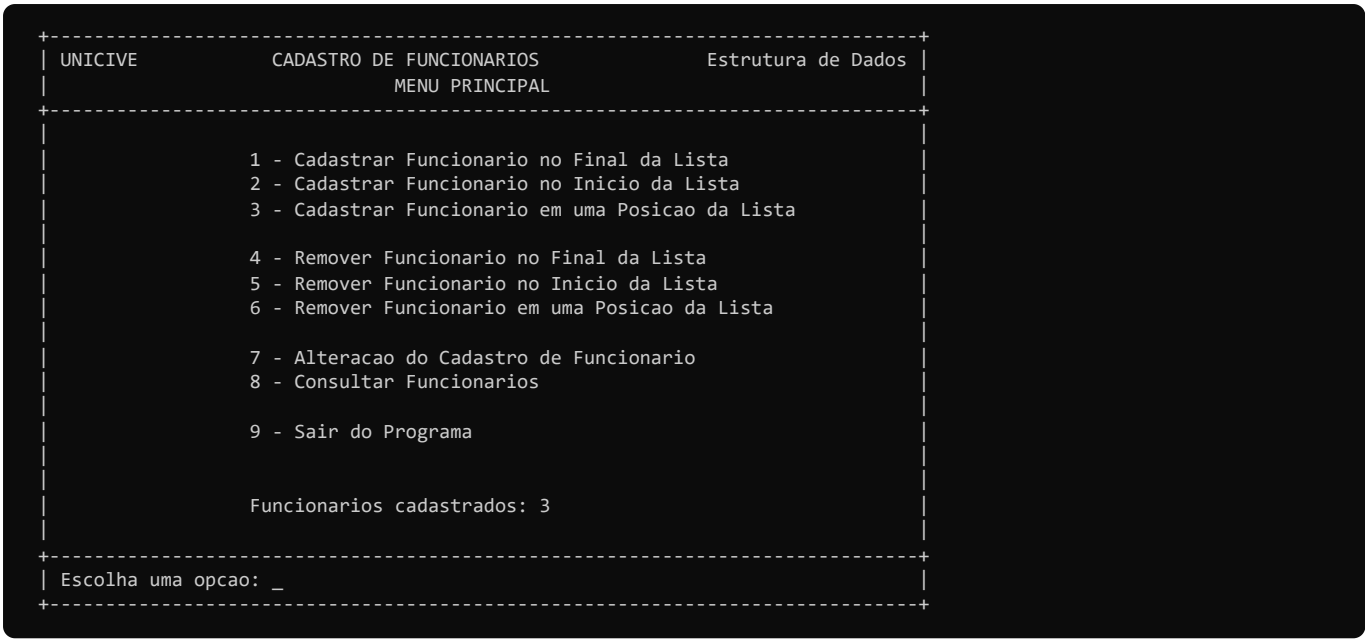
    gotoxy(20, 20);
    printf("Funcionarios cadastrados: %d", contar_funcionarios(L));

    gotoxy(03, 23);
    printf("Escolha uma opcao: ");
    fflush(stdin);
    scanf("%d", &opcao);

    return opcao;
}
```

`menu(TipoLista *L)`

Desenha a moldura, escreve as 9 opções (com linhas em branco separando os grupos cadastrar / remover / alterar-consultar / sair) e mostra quantos funcionários existem, usando `contar_funcionarios`. Lê a opção e a **devolve** para o `main` decidir o que fazer. A variável começa com `-1` para que uma entrada inválida (uma letra, por exemplo) caia no `default` do `switch`.



Simulação do menu principal, montada com as mesmas coordenadas de `gotoxy` usadas no código (exemplo com 3 funcionários).

Registro_Funcionarios.c — linhas 848–901

```c
// Programa principal
int main()
{
    int opcao;
    TipoLista L;

    // Inicializa a lista vazia e carrega os funcionários gravados em disco
    inicializar_lista(&L);
    Ler_Arquivo(&L);

    do
    {
        opcao = menu(&L);

        switch (opcao)
        {
        case 1:
            cadastrar(&L, 1);
            break;
        case 2:
            cadastrar(&L, 2);
            break;
        case 3:
            cadastrar(&L, 3);
            break;
        case 4:
            remover(&L, 1);
            break;
        case 5:
            remover(&L, 2);
            break;
        case 6:
            remover(&L, 3);
            break;
        case 7:
            alterar(&L);
            break;
        case 8:
            consultar(&L);
            break;
        case 9:
            Gravar(&L);
            liberar_lista(&L);
            gotoxy(01, 25);
            break;
        default:
            mensagem("OPCAO INVALIDA... Pressione uma tecla.");
            break;
        }

    } while (opcao != 9);

    return 0;
}
```

`main()` — onde tudo começa

1. `TipoLista L;` — a única variável de lista do programa. Ela vive no `main` e é passada por endereço (`&L`) para todas as funções.
2. `inicializar_lista(&L)` e `Ler_Arquivo(&L)` — começa vazia e recupera os dados salvos (ou avisa que o arquivo não foi encontrado).
3. `do { ... } while (opcao != 9)` — o **laço principal**: mostra o menu, executa a opção e repete até o usuário escolher 9.
4. O `switch` direciona cada opção. Note como as opções 1/2/3 e 4/5/6 chamam a mesma função mudando só o `tipo` . O `break` em cada `case` impede que a execução "escorra" para o caso seguinte.
5. **Opção 9:** `Gravar(&L)` grava o arquivo e mostra a mensagem de sucesso ou erro; `liberar_lista(&L)` libera a memória; e `gotoxy(01, 25)` posiciona o cursor na linha 25 (abaixo da moldura), para que o prompt do Windows não apareça por cima da tela.
6. `default` : qualquer outro número exibe "OPCAO INVALIDA".
7. `return 0;` — avisa ao sistema operacional que o programa terminou sem erros.

Fluxo completo de uma execução

Momento	O que acontece
Início	`main` → `inicializar_lista` → `Ler_Arquivo` (vários `inserir_final`)
Cada volta	`menu` → `contar_funcionarios` → opção escolhida
Ex.: opção 3	`cadastrar(&L, 3)` → `pesquisa` → `ler_funcionario` → `inserir_posicao` → `buscar_posicao` → `novo_item` → `malloc`
Ex.: opção 8	`consultar(&L)` → `contar_funcionarios` → `exibir_funcionario` → `getch` (P: `p->proximo` · V: `buscar_posicao` · S: sai)
Ex.: opção 4	`remover(&L, 1)` → `buscar_posicao` → `exibir_funcionario` → `remover_final` → `free`
Fim	opção 9 → `Gravar` (`fopen "wb"` + vários `fwrite`) → `liberar_lista` → `return 0`

17. Resumo, complexidade e pontos de atenção

Custo de cada operação

"O(1)" significa tempo constante (não depende do tamanho da lista); "O(n)" significa que, no pior caso, é preciso percorrer os *n* nós.

Operação	Custo	Motivo
`inserir_inicio`	O(1)	Só mexe em `Primeiro`.
`inserir_final`	O(1)	O ponteiro `Ultimo` evita o percurso.
`inserir_posicao`	O(n)	Precisa andar até a posição (e conta a lista).
`remover_inicio`	O(1)	Só mexe em `Primeiro`.
`remover_final`	O(n)	Precisa achar o penúltimo — não existe ponteiro para trás.
`remover_posicao`	O(n)	Precisa andar até o anterior.
`pesquisa`, `contar_funcionarios`	O(n)	Percorrem a lista.
Consulta: tecla P (próximo)	O(1)	Só segue a seta `p->proximo`.
Consulta: tecla V (voltar)	O(n)	Recomeça do `Primeiro` com `buscar_posicao` — não existe ponteiro para trás.
`Ler_Arquivo`, `Gravar`	O(n)	Leem/gravam um registro por funcionário.

Para pensar

Se cada nó tivesse também um ponteiro `anterior` (lista **duplamente** encadeada), `remover_final` e a tecla V da consulta passariam a ser O(1). Qual seria o custo disso em memória e em quantidade de ponteiros para atualizar em cada inserção?

Pontos de atenção (para estudar e discutir)

1. `fflush(stdin)`

Pelo padrão da linguagem C, `fflush` só é definido para fluxos de **saída**; em `stdin` o comportamento é indefinido. No Windows com MinGW/MSVC antigos ele limpa o buffer do teclado e por isso funciona aqui, mas em outros compiladores/sistemas pode não ter efeito. A forma portátil é consumir os caracteres até o `'\n'` com um pequeno laço de `getchar()`.

2. Tamanho de `dt_nasc`

O vetor tem 10 posições e `ler_texto` lê até 9 caracteres. O formato pedido (`dd/mm/aa`, 8 caracteres) cabe. Se o usuário digitar o ano com 4 dígitos (`dd/mm/aaaa`, 10 caracteres), a data será cortada. O mesmo raciocínio vale para os outros campos de texto.

3. `resp` e o `continue` em `cadastrar` / `remover` — CORRIGIDO

Na versão anterior, uma posição inválida na primeira volta fazia o `continue` testar `while (resp == 1)` com `resp` ainda sem valor (lixo de memória). Agora as duas funções fazem `resp = 1;` antes do `continue`, e a tela sempre se repete. Lição geral: **toda variável lida numa condição precisa ter recebido um valor antes** — em todos os caminhos possíveis até ali.

4. Gravação só ao sair

Os dados vão para o disco apenas na opção 9. Se a janela for fechada no "X" ou o programa travar, tudo o que foi feito na sessão se perde.

5. Arquivo binário depende do compilador

`fwrite` grava os bytes da struct como estão na memória (incluindo possíveis bytes de alinhamento). Se o tamanho da struct mudar (por exemplo, aumentar um campo) ou o programa for compilado de forma diferente, um `funcionarios.dat` antigo será lido incorretamente.

6. Validação do `scanf`

O retorno do `scanf` (quantos valores leu) não é verificado. O código se protege inicializando as variáveis (`opcao = -1`, `resp = 0`, `posicao = 0`), o que já evita a maioria dos problemas com entradas não numéricas.

Conceitos que este programa exercita

- `struct` e `typedef`
- Estrutura autorreferenciada (nó)
- Ponteiros, `->`, `&` e `*`
- Passagem por valor × por referência
- Alocação dinâmica (`malloc` / `free`)
- Lista simplesmente encadeada com `Primeiro` / `Ultimo`
- Percurso, inserção e remoção em qualquer posição
- Leitura segura de strings (`fgets` + `strcspn`)
- Arquivos binários (`fread` / `fwrite`)
- Interface de console com `gotoxy`